



RAVI KISHAN
CSE(AIML)

Dataset link:

MNIST:

<https://keras.io/api/data-sets/mnist/>

DEEP LEARNING PROJECT REPORT

Assignment 1: Artificial Neural Network (ANN) for Digit Classification

Introduction

Artificial Neural Network (ANN) is a deep learning model inspired by the human brain. It consists of interconnected neurons organized into layers. ANNs learn patterns from data by adjusting weights during training.

For image classification, an ANN receives pixel values as input and predicts the correct class using multiple hidden layers. In this assignment, the MNIST handwritten digit dataset is used to train the ANN.

Objective

The objective of this assignment is to build and train an Artificial Neural Network (ANN) to classify handwritten digits (0–9) using the MNIST dataset. The model is evaluated using accuracy and loss, and predictions are made on unseen test images.

Dataset Description

The MNIST dataset is one of the most widely used benchmark datasets for handwritten digit recognition.

- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Number of Classes: 10 (Digits 0–9)

- Image Type: Grayscale

Import Libraries:

Before building the model, required libraries are imported. TensorFlow is used for deep learning, NumPy for numerical operations, and Matplotlib for visualization.

```
[2]
✓ 0s import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

Dataset Loaded

```
[3]
✓ 0s (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

v Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
```

Check Shape

```
[4]
✓ 0s print("Training Images :", X_train.shape)
print("Training Labels :", y_train.shape)

print("Testing Images :", X_test.shape)
print("Testing Labels :", y_test.shape)

v Training Images : (60000, 28, 28)
Training Labels : (60000,)
Testing Images : (10000, 28, 28)
Testing Labels : (10000,)
```

Display Sample Images



Normalize Data

```
] Os  
X_train = X_train / 255.0  
X_test = X_test / 255.0
```

Build ANN Model

The ANN architecture contains:

- Input Layer
- Flatten Layer
- Hidden Layer 1 (128 neurons, ReLU)
- Hidden Layer 2 (64 neurons, ReLU)
- Output Layer (10 neurons, Softmax)

ReLU introduces non-linearity, while Softmax converts outputs into class probabilities.

```
[7]
✓ 0s
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequ
super().__init__(**kwargs)
```

Model Summary

```
[8]
✓ 0s
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Compile Model

```
[9]
✓ 0s
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Train Model

```
[10]
✓ 39s history = model.fit(
    X_train,
    y_train,
    epochs=10,
    validation_split=0.2
)

Epoch 1/10 1500/1500 ————— 5s 3ms/step - accuracy: 0.9215 - loss: 0.2678 - val_accuracy: 0.9607 - val_loss: 0.1327
Epoch 2/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9666 - loss: 0.1098 - val_accuracy: 0.9650 - val_loss: 0.1113
Epoch 3/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9764 - loss: 0.0754 - val_accuracy: 0.9712 - val_loss: 0.1003
Epoch 4/10 1500/1500 ————— 4s 3ms/step - accuracy: 0.9828 - loss: 0.0550 - val_accuracy: 0.9698 - val_loss: 0.0968
Epoch 5/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9860 - loss: 0.0449 - val_accuracy: 0.9711 - val_loss: 0.0955
Epoch 6/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9883 - loss: 0.0365 - val_accuracy: 0.9732 - val_loss: 0.0995
Epoch 7/10 1500/1500 ————— 4s 3ms/step - accuracy: 0.9904 - loss: 0.0289 - val_accuracy: 0.9724 - val_loss: 0.1082
Epoch 8/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9922 - loss: 0.0232 - val_accuracy: 0.9768 - val_loss: 0.0993
Epoch 9/10 1500/1500 ————— 4s 2ms/step - accuracy: 0.9923 - loss: 0.0237 - val_accuracy: 0.9693 - val_loss: 0.1318
Epoch 10/10 1500/1500 ————— 4s 3ms/step - accuracy: 0.9939 - loss: 0.0180 - val_accuracy: 0.9709 - val_loss: 0.1366
```

Evaluate Model

```
[11]
✓ 0s loss, accuracy = model.evaluate(X_test, y_test)

print("Loss :", loss)

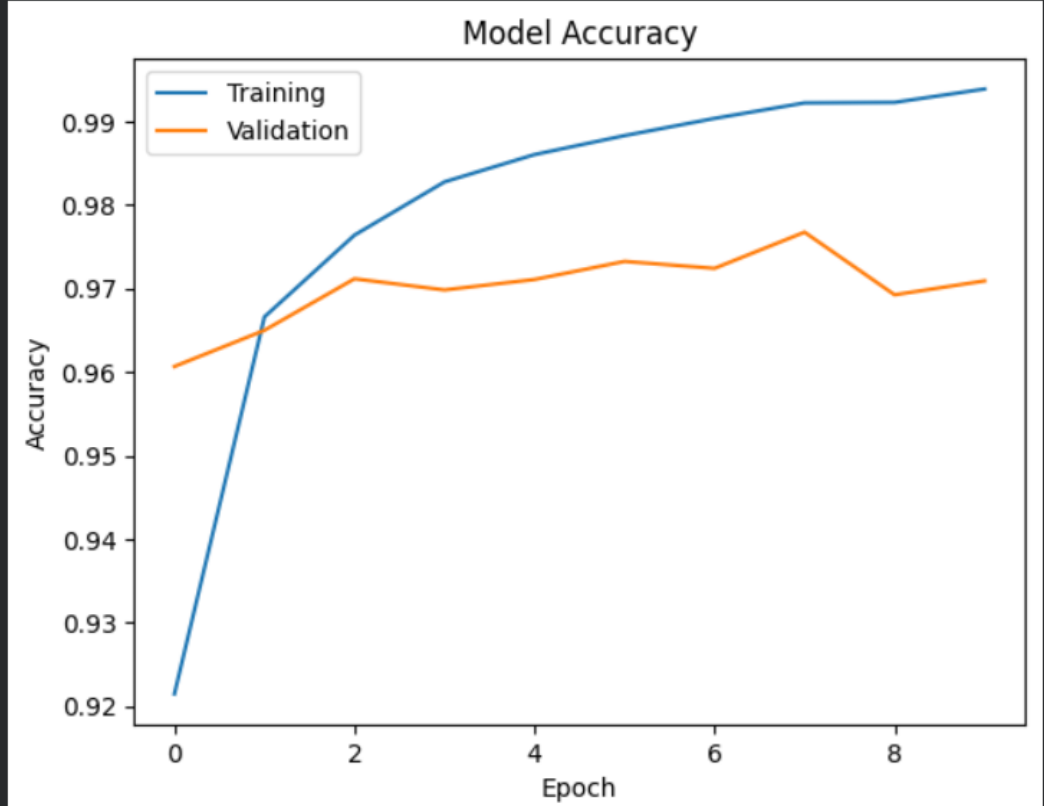
print("Accuracy :", accuracy)

313/313 ————— 1s 1ms/step - accuracy: 0.9727 - loss: 0.1261
Loss : 0.12611474096775055
Accuracy : 0.9726999998092651
```

Accuracy Graph

[12]
✓ 0s

```
plt.plot(history.history['accuracy'])  
  
plt.plot(history.history['val_accuracy'])  
  
plt.title("Model Accuracy")  
  
plt.xlabel("Epoch")  
  
plt.ylabel("Accuracy")  
  
plt.legend(['Training', 'Validation'])  
  
plt.show()
```

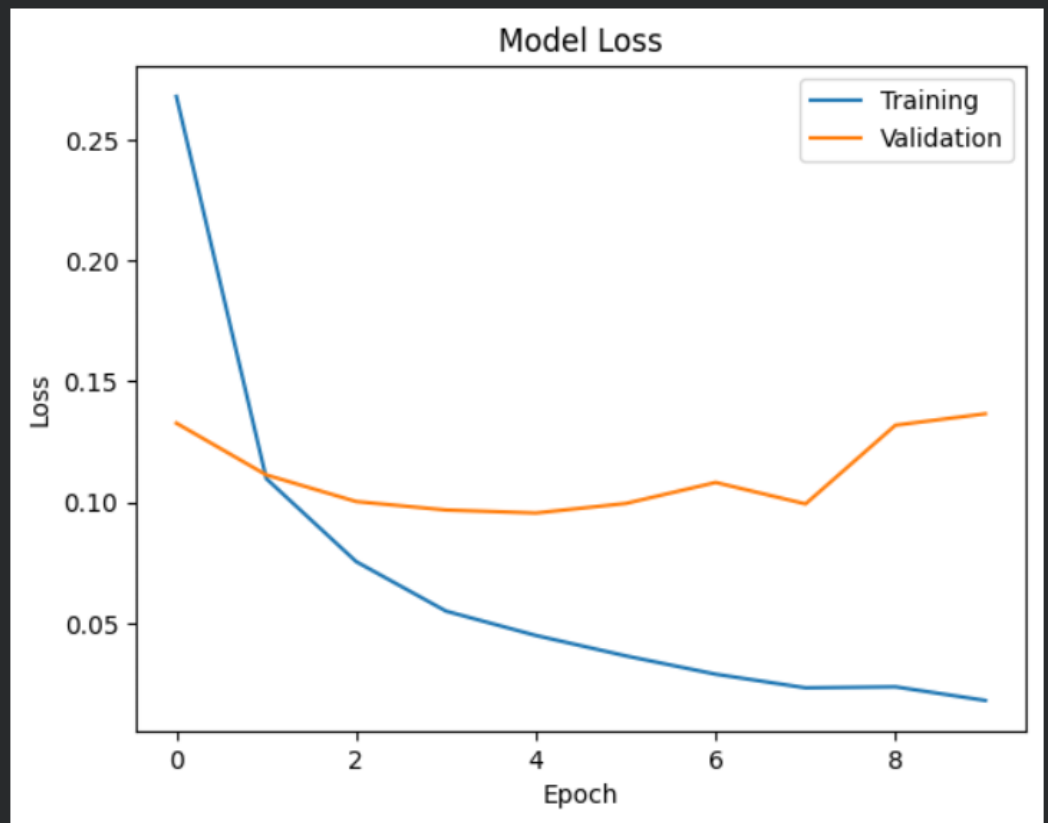


Loss Graph

```
plt.ylabel("Loss")

plt.legend(['Training', 'Validation'])

plt.show()
```



Predict 10 Test Images

```
predictions = model.predict(X_test)

plt.figure(figsize=(12,6))

for i in range(10):

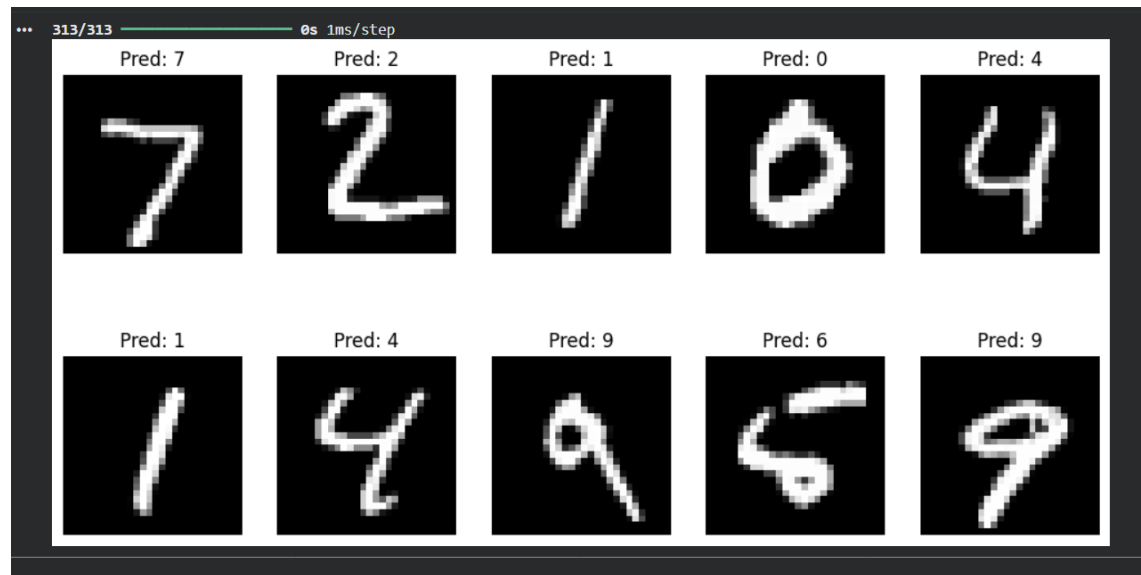
    plt.subplot(2,5,i+1)

    plt.imshow(X_test[i], cmap="gray")

    plt.title("Pred: {}".format(np.argmax(predictions[i])))

    plt.axis("off")

plt.show()
```

Conclusion

An Artificial Neural Network (ANN) was successfully developed and trained using the MNIST dataset. The model effectively classified handwritten digits from 0 to 9 with high accuracy. The training and validation accuracy increased over epochs, while the loss decreased, indicating that the model learned meaningful patterns from the data. The prediction results on test images demonstrate the effectiveness of ANN for handwritten digit recognition.

